WNDRVR
SUPPORT NETWORK

VXWORKS 7 RENESAS_RCAR_H3 BSP SUPPLEMENT GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 renesas_rcar_h3 BSP Supplement Guide, SR0650

25 August 2020

1. INTRODUCTION

1. Introducing the renesas_rcar_h3 BSP

This document describes the features of the renesas_rcar_h3 BSP, which is designed to run on the Renesas R-Car H3SiP System Evaluation Board Salvator-X RTP0RC7795SIPB0011S-S and Salvator-XS RTP0RC7795SIPB0012SA03 development boards.

This document must be read in conjunction with the BSP readme file that relates to your hardware board. Refer to either *rcar_h3sip_salvator_x_readme.md* or *rcar_h3sip_salvator_xs_readme.md*.

Note: The Helix Platform is only supported on the Salvator-XS board, not the Salvator-X.

While the BSP readme file guides you through a simple target boot process, this document gives a more complete description of the BSP and its capabilities.

Before using a board with VxWorks®, verify that the board runs in the factory configuration, and check all target board connections. Refer to the board vendor documentation for assistance.

2. Introducing the Boot Process

This BSP supports booting the target board from flash using U-Boot which loads either the VxWorks kernel directly or the boot application.

Three approaches can be taken when booting the target board.

Table 1. Target Boot Strategies

Boot Loader	Image File Loaded By Boot Loader	Image File Data Model	Image File Name	Independent or Embedded DTB
U-Boot	VxWorks kernel	64-bit	uVxWorks	Independent or Embedded
U-Boot	VxWorks kernel	64-bit	vxWorks.bin	Embedded
U-Boot	Boot Application	64-bit	vxWorks	Embedded

3. Configure Helix Development Environment

Initialise the workstation command shell for VxWorks board support package (BSP) configuration and project build for Helix.

If your workstation runs Windows, refer to Setting Up a Windows Development Environment for Helix on page 2 for instructions.

If your workstation runs Linux, refer to Setting Up a Linux Development Environment for Helix on page 2 for instructions.

Setting Up a Windows Development Environment for Helix on page 2
Setting Up a Linux Development Environment for Helix on page 2

WNDRVR

3.1. Setting Up a Windows Development Environment for Helix

Before You Begin

Before you begin you must install the Helix Platform on your host workstation.

About This Task

These steps configure the workstation command terminal environment variables to allow for Helix-VxWorks command-line interface (CLI) development tasks to be performed.

Procedure

1. Change into the Wind River directory.

```
C:\> cd WindRiver
```

2. Add Wind River build tools to the command terminal path.

```
C:\WindRiver> wrenv -p helix
Microsoft Windows [Version 6.1.7601]
Copyright © 2009 Microsoft Corporation. All rights reserved.

C:\WindRiver>
```

3. Change into the Workbench workspace directory.

```
C:\WindRiver> cd workspace
C:\WindRiver\workspace>
```

Parent topic: Configure Helix Development Environment on page 1

3.2. Setting Up a Linux Development Environment for Helix

Before You Begin

Before you begin you must install the Helix Platform on your host workstation.

About This Task

These steps configure the workstation command terminal environment variables to allow for Helix-VxWorks command-line interface (CLI) development tasks to be performed.

Procedure

1. Change into the Wind River directory.

```
~/home/user$ cd WindRiver
```


WNDRVR

2. Add Wind River build tools to the command terminal path.

```
~/home/user/WindRiver$ ./wrenv.linux -p helix
~/home/user/WindRiver$
```

3. Create the workspace directory.

```
~/home/user/WindRiver$ mkdir workspace
~/home/user/WindRiver$
```

4. Change into the workspace directory.

```
~/home/user/WindRiver$ cd workspace
~/home/user/WindRiver/workspace$
```

Parent topic: Configure Helix Development Environment on page 1

4. Configure Development Environment

Initialise the workstation command shell for VxWorks Board Support Package (BSP) configuration and project build.

If your workstation runs Windows, refer to Setting Up a Windows Development Environment on page 3 for instructions.

If your workstation runs Linux, refer to Setting Up a Linux Development Environment on page 4 for instructions.

Setting Up a Windows Development Environment on page 3 The Windows command-line interface must be configured correctly for use with VxWorks.
Setting Up a Linux Development Environment on page 4 The Linux command-line interface must be configured correctly for use with VxWorks.

4.1. Setting Up a Windows Development Environment

The Windows command-line interface must be configured correctly for use with VxWorks.

Before You Begin

Before you begin you must install Wind River VxWorks 7 on your host workstation.

About This Task

These steps configure the workstation command terminal environment variables to allow for VxWorks command-line interface (CLI) development tasks to be performed.

Procedure

1. Change into the Wind River directory.

```
C:\> cd WindRiver
```

2. Add Wind River build tools to the command terminal path.

```
C:\WindRiver> wrenv -p vxworks-7
Microsoft Windows [Version 6.1.7601]
Copyright © 2009 Microsoft Corporation. All rights reserved.

C:\WindRiver>
```

3. Change into the Workbench workspace directory.

```
C:\WindRiver> cd workspace
C:\WindRiver\workspace>
```

Parent topic: Configure Development Environment on page 3

4.2. Setting Up a Linux Development Environment

The Linux command-line interface must be configured correctly for use with VxWorks.

Before You Begin

Before you begin you must install Wind River VxWorks 7 on your host workstation.

About This Task

These steps configure the workstation command terminal environment variables to allow for VxWorks command-line interface (CLI) development tasks to be performed.

Procedure

1. Change into the Wind River directory.

```
~/home/user$ cd WindRiver
```

2. Add Wind River build tools to the command terminal path.

```
~/home/user/WindRiver$ ./wrenv.linux -p vxworks-7
~/home/user/WindRiver$
```

3. Create the workspace directory.

```
~/home/user/WindRiver$ mkdir workspace
~/home/user/WindRiver$
```

4. Change into the workspace directory.

```
~/home/user/WindRiver$ cd workspace
~/home/user/WindRiver/workspace$
```

Parent topic: Configure Development Environment on page 3

2. CONFIGURATION AND SETUP

1. How to Rebuild U-Boot

Refer to the appropriate readme file for instructions on retrieving, rebuilding and deploying U-Boot in target board flash memory. Refer to either *rcar_h3sip_salvator_x_readme.md* or *rcar_h3sip_salvator_xs_readme.md*.

2. Device Tree and Device Tree Blob Files

This board support package (BSP) uses device tree nodes to describe the device specific information on the board. The supported drivers will get configuration data from the device tree source file (suffixed .dts) that is located in the BSP directory.

For a description of the device tree source file structure, refer to the device tree specification.

The device tree source file can be used in your VxWorks projects in either embedded or independent mode. Use **renesas-salvator-x.dts** when your project is intended for the Salvator-X board and **renesas-salvator-xs.dts** when the board is the Salvator-XS. In embedded mode, the device tree blob is embedded in the VxWorks image when the VIP is built. The VIP parameter DTS_FILE must be set to point to the appropriate DTS file in the BSP directory.

3. Workstation Serial Driver

Serial connectivity is provided through the micro-AB USB connector via the USB to UART bridge CP2102.

Because serial connectivity is provided through a USB connector, a USB driver must be available on the host workstation to support this connection.

On a Windows and Linux host, you must install the USB driver before you use the serial connection. Locate the drivers on the Silicon Labs website at https://www.silabs.com/products/development-tools/software/usb-to-uart-bridge-vcp-drivers.

Note:

Once the driver is installed on Windows, the host will register a virtual COM port once the USB cable is attached between the workstation and the target board. Attach the serial terminal to this virtual COM port with the following configuration:

- Baud Rate: 115200
- Data: 8 bit
- Parity: None
- Stop: 1 bit
- Flow Control: None

3. VXWORKS PROJECTS

1. Project Creation Workflow

A VxWorks source build (VSB) project must always be created before the VxWorks image project (VIP).

When creating a VSB using the renesas_rcar_h3 BSP, Wind River recommends you also specify the name of the Board Support Page (BSP) at VSB creation time, as this automatically configures the VSB with the necessary VxWorks layers.

When creating the VSB, you have the option of specifying the CPU instead of the BSP. If you do this, you must remember to add the correct VxWorks layers to the VSB before building the project. Refer to Creating a Uniprocessor and CPU Based VxWorks Source Build on page 6 for details on how to create the VSB project and add VxWorks layers to it.

From the command-line, you specify the BSP in the VSB creation command by adding -bsp renesas_rcar_h3. You specify the 32-bit CPU in the VSB creation command by adding -cpu ARMARCH8A to the VSB creation command.

Note: 32-bit VSB is unsupported by this BSP.

Note: The project creation workflow can also be achieved within the Workbench Eclipse IDE. For details, refer to *Workbench 4 Getting Started: Workbench Project Overview*

2. Processing Environments and Data Models

When building projects with this Board Support Package (BSP) there are two processing environments and one data model.

- ARM v8A - 64-bit arm CPU uniprocessor

Note: All 64-bit arm projects are also referred to as using ARM v8A, or ARMARCH8A build spec.

Table 1. Processing Environment and Data Model VSB Options

Data Model Option	Parameter Specified During VSB Creation
ARM v8 64-bit Data Width	-lp64 (default)
Symmetric Multiprocessor	-smp (default)
Uniprocessor	-up

Note: When creating a VxWorks Source Build (VSB), it defaults to symmetric multiprocessor on creation.

3. Creating a Uniprocessor and CPU Based VxWorks Source Build

Before You Begin

Ensure your VxWorks development environment is correctly configured. Refer to Configure Development Environment on page 3.

Procedure

1. Create the VSB.

```
C:\WindRiver\workspace> vxprj vsb create -force -lp64 -up -cpu ARMARCH8A -S myVSB
C:\WindRiver\workspace> cd myVSB
C:\WindRiver\workspace\myVSB> vxprj vsb add RENESAS_RCAR
C:\WindRiver\workspace\myVSB> vxprj vsb config -s -add _WRS_CONFIG_MULTI_CLUSTERS=y
```

2. Build the VSB.

```
C:\WindRiver\workspace\myVSB> vxprj build
```

Note: To build the VSB with parallelism enabled, you must use the command make. For example, if the workstation has eight cores, you build the project using all of these cores with the following command:

```
C:\WindRiver\workspace\myVSB> make JOBS=8
```

4. Creating a Boot Application Project

About This Task

In these steps you will build a VxWorks boot application image.

Procedure

1. Create a uniprocessor VxWorks source build (VSB).
   1. Create the VSB.

```
C:\WindRiver\workspace> vxprj vsb create -lp64 -up -bsp renesas_rcar_h3 -S myVSB
C:\WindRiver\workspace> cd myVSB
```

   2. Build the VSB.

```
C:\WindRiver\workspace\myBootApp> vxprj build
```

2. Create a VxWorks image project (VIP) with the bootapp profile selected.

```
C:\WindRiver\workspace> vxprj vip create -vsb myVSB -profile PROFILE_BOOTAPP renesas_rcar_h3 my
BootApp
C:\WindRiver\workspace> cd myBootApp
```

3. Select the hardware board you will run the boot application on.

Option	Description
Salvator-X board	`C:\WindRiver\workspace\myBootApp> vxprj component add INCLUDE_RENESAS_SALVATOR_X`
Salvator-XS board	`C:\WindRiver\workspace\myBootApp> vxprj component add INCLUDE_RENESAS_SALVATOR_XS`

4. Build the boot application VIP.

```
C:\WindRiver\workspace\myBootApp> vxprj build vxWorks.bin
```

5. The VxWorks Image Project

Refer to the readme for instructions on creating a VxWorks image project (VIP) for your board. Refer to either *rcar_h3sip_salvator_x_readme.md* or *rcar_h3sip_salvator_xs_readme.md*.

Creating a VxWorks Kernel Image with Embedded DTB on page 8
Creating a Helix Guest VxWorks Kernel Image on page 10

5.1. Creating a VxWorks Kernel Image with Embedded DTB

Before You Begin

These steps assume you have already created a VxWorks source build (VSB) configured in either uniprocessor or symmetric multiprocessor configuration.

Procedure

1. Create the VxWorks Image Project (VIP).

```
C:\WindRiver\workspace> vxprj vip create -vsb myVSB -profile PROFILE_DEVELOPMENT renesas_rcar_h3 myVIP
C:\WindRiver\workspace> cd myVIP
```

> **Note:** If you do not specify the VIP creation parameter -profile PROFILE_DEVELOPMENT, then you must remember to manually include the network END driver component into the VIP, **INCLUDE_END**.

2. Select the hardware board you will run the boot application on.

Option	Description
Salvator-X board	`C:\WindRiver\workspace\myVIP vxprj component add INCLUDE_RENESAS_SALVATOR_X`
Salvator-XS board	`C:\WindRiver\workspace\myVIP vxprj component add INCLUDE_RENESAS_SALVATOR_XS`

3. Select an embedded Device Tree Blob (DTB) file.

```
C:\WindRiver\workspace\myVIP> vxprj vip component add myVIP INCLUDE_STANDALONE_DTB
```

4. Select the device tree source file for your target board.

Option	Description
Salvator-X board	`C:\WindRiver\workspace\myVIP> vxprj vip parameter set DTS_FILE "\"renesas-salvator-x.dts"\"`
Salvator-XS board	`C:\WindRiver\workspace\myVIP> vxprj vip parameter set DTS_FILE "\"renesas-salvator-xs.dts"\"`

Note: If you are using a linux workstation, the command must be constructed as follows:

```
~$ vxprj vip parameter set DTS_FILE '"renesas-salvator-x.dts"'
```

5. Configure the VxWorks boot line for the BSP. Refer to either *rcar_h3sip_salvator_x_readme.md* or *rcar_h3sip_salvator_xs_readme.md* for details.
6. Set the Target Board MAC address in the Device Tree Source (DTS) file. This step is only required if you decide not to set the MAC address in the boardEndMacGet( ) function, as described in Adding the Ethernet Address in boardEndMacGet() on page 14.
   1. Edit the correct device tree source file for your target board.

      If you are using the Salvator-X board, open **renesas-salvator-x.dts**. Otherwise, open **renesas-salvator-xs.dts**.

   2. Locate the network device:

```
&avb
    {
    /* U-Boot bootm command will update local-mac-address in standalone DTB */

    local-mac-address = [ 2E 09 0A 00 00 00 ];
```

   3. Edit the **local-mac-address** property.

```
    local-mac-address = [ 00 0A 35 11 22 33 ];
```

> **Note:** Replace the specified MAC address with your target board MAC address.

7. Build the VIP.

```
C:\WindRiver\workspace\myVIP> vxprj build
```

Parent topic: The VxWorks Image Project on page 8

5.2. Creating a Helix Guest VxWorks Kernel Image

Before You Begin

These steps assume you have already created a VxWorks source build (VSB) configured in either uniprocessor or symmetric multiprocessor configuration.

Procedure

1. Create the VxWorks Image Project (VIP).

```
C:\WindRiver\workspace> vxprj vip create -vsb myVSB -profile PROFILE_DEVELOPMENT renesas_rcar_h
3 myVIP
C:\WindRiver\workspace> cd myVIP
```

> **Note:** If you do not specify the VIP creation parameter -profile PROFILE_DEVELOPMENT, then you must remember to manually include the network END driver component into the VIP, **INCLUDE_END**.

2. Select the Salvator-XS hardware board you will run the boot application on.

Option	Description
Salvator-XS board	`C:\WindRiver\workspace\myVIP vxprj comp onent add INCLUDE_RENESAS_SALVATOR_XS`

3. Select an embedded Device Tree Blob (DTB) file.

```
C:\WindRiver\workspace\myVIP> vxprj vip component add myVIP INCLUDE_STANDALONE_DTB
```

4. Select the device tree source file for your target board.

Option	Description
Salvator-XS board	`C:\WindRiver\workspace\myVIP> vxprj vi p parameter set DTS_FILE "\"renesas-sal vator-xs.dts"\"`

WNDRVR

Note: If you are using a linux workstation, the command must be constructed as follows:

```
~$ vxprj vip parameter set DTS_FILE '"renesas-salvator-xs.dts"'
```

5. Configure the VxWorks boot line for the BSP. Refer to *rcar_h3sip_salvator_xs_readme.md* for details.
6. Set the Target Board MAC address in the Device Tree Source (DTS) file. This step is only required if you decide not to set the MAC address in the boardEndMacGet( ) function, as described in Adding the Ethernet Address in boardEndMacGet() on page 14.
   1. Edit the correct device tree source file for your target board.

      Open **renesas-salvator-xs.dts**.

      2. Locate the network device:

      ```
      &avb
          {
          /* U-Boot bootm command will update local-mac-address in standalone DTB */

          local-mac-address = [ 2E 09 0A 00 00 00 ];
      ```

      3. Edit the **local-mac-address** property.

      ```
          local-mac-address = [ 00 0A 35 11 22 33 ];
      ```

      Note: Replace the specified MAC address with your target board MAC address.

7. Build the VIP.

```
C:\WindRiver\workspace\myVIP> vxprj build
```

Note: For further details about how to configure a Helix guest operating system, refer to the Helix Platform Configuration and Build Guide.

Parent topic: The VxWorks Image Project on page 8

4. TARGET BOOT WORKFLOWS

1. Downloading a Binary VxWorks Image With Embedded DTB from U-Boot

Before You Begin

These steps assume you have created and built a VxWorks image project (VIP) with embedded DTB. Refer to Creating a VxWorks Kernel Image with Embedded DTB on page 8 for more details.

Procedure

1. Power up the target board to the U-Boot shell.
2. Load the VxWorks image into target memory.

```
=> tftp 0x48100000 vxWorks.bin
```

3. Run the VxWorks kernel.

```
=> go 0x48100000
```

2. Downloading a uVxWorks Image With Embedded DTB from U-Boot

Before You Begin

These steps assume you have created and built a VxWorks image project (VIP) with embedded DTB. Refer to Creating a VxWorks Kernel Image with Embedded DTB on page 8 for more details.

Procedure

1. Power up the target board to the U-Boot shell.
2. Load the VxWorks image into target memory.

```
=> tftp 0x50100000 uVxWorks
```

3. Run the VxWorks kernel.

```
=> bootm 0x50100000
```

3. Downloading a Boot Application Image With Embedded DTB from U-Boot

Before You Begin

These steps assume you have created and built a boot application. Refer to Creating a Boot Application Project on page 7 for more details.

Procedure

1. Power up the target board to the U-Boot shell.
2. Load the VxWorks image into target memory.

```
=> tftp 0x4a000000 vxWorks.bin
```

3. Run the VxWorks kernel.

```
=> go 0x4a000000
```

4. Boot the VxWorks Image Using the Boot Application

The boot application can only boot a traditional VxWorks ELF image configured with an embedded DTB. For further details on creation of this VxWorks image project, refer to Creating a VxWorks Kernel Image with Embedded DTB on page 8.

The boot application assumes that an FTP server is running on the host workstation. Use the following parameters to FTP the VxWorks image from the boot application on the target.

Note: Replace **inet on ethernet (e)**, **host inet (h)**, **user (u)** and **ftp password (pw)** with the settings appropriate for your network and FTP server.

```
boot device          : etherAvb
unit number          : 0
processor number     : 0
host name            : host
file name            : vxWorks
inet on thernet (e)  : 192.168.1.100
host inet (h)        : 192.168.1.1
user (u)             : target
ftp password (pw)    : vxTarget
flags (f)            : 0x0
```

5. DEVICE-SPECIFIC INFORMATION

1. Network and Ethernet (MAC) Configuration

The Salvator-X and Salvator-XS boards embed a gigabit ethernet transceiver, Micrel KSZ9031RNXVA, connected to the EtherAVB of the R-CarH3-SiP.

To enable the ethernet controller, you can follow one of two methods:

1. Specify **PROFILE_DEVELOPMENT** when creating the VxWorks image project (VIP). The related components are added and the network is configured.
2. Create the VIP with the default profile and add the component **INCLUDE_END** to it.

Adding the Ethernet Address in boardEndMacGet() on page 14
Configuring the Ethernet Address from the U-Boot Console on page 15U-Boot will usually allow the target board ethernet (MAC) address to be updated from the U-Boot serial console.

1.1. Adding the Ethernet Address in boardEndMacGet()

Before You Begin

These steps assume you have created a VxWorks image project (VIP) called **myVIP**.

About This Task

These steps declare a function, boardEndMacGet( ), which has four parameters defined as follows:

ifName

A pointer to a string defining the network interface name of the network controller.

ifUnit

The network interface index number.

ifMacAddr

A pointer to the memory storing the MAC address.

ifMacAddrLen

The length of the MAC address.

Procedure

1. Locate the source file sysLib.c( ).

   This is usually located in the workspace at **myVIP/***bsp_name_w_x_y_z***/sysLib.c**.

   Note: The BSP layer version number is represented by *w_x_y_z*.

2. Identify and modify the structure **boardFuncTbl**.

Modify the **boardFuncTbl** structure as follows:

```
BOARD_FUNC_TBL boardFuncTbl =
{
    ….
    /* .endMacGet = */ boardEndMacGet
```

3. Create a forward declaration for function boardEndMacGet( ).

```
/* forward declarations */

LOCAL STATUS boardEndMacGet
    (
    char *          ifName,
    unsigned int    ifUnit,
    UINT8 *         ifMacAddr,
    unsigned int    ifMacAddrLen,
    );
```

4. Create the function boardEndMacGet( ).

   Add the function to the bottom of the file.

```
STATUS boardEndMacGet
    (
    char *          ifName,
    unsigned int    ifUnit,
    UINT8 *         ifMacAddr,
    unsigned int    ifMacAddrLen
    )
    {
    UINT8 macAddr[6] = {0x00, 0x0A, 0x35, 0x11, 0x22, 0x33};

    if ((ifName == NULL) || (ifMacAddr == NULL))
        return ERROR;

    bcopy(macAddr, ifMacAddr, ifMacAddrLen);

    return OK;
    }
```

Note: Update the array **macAddr** directly to reflect the actual ethernet address (MAC address) of your target board. Alternatively, code this function to retrieve the address from a memory device on the board and then set the array **macAddr** to it.

Parent topic: Network and Ethernet (MAC) Configuration on page 14

1.2. Configuring the Ethernet Address from the U-Boot Console

U-Boot will usually allow the target board ethernet (MAC) address to be updated from the U-Boot serial console.

Refer to your target documentation to check if U-Boot supports setting the *ethaddr* variable, and also whether you can save the environment variables from the U-Boot console. If it does then VxWorks will inherit the ethernet address you set from U-Boot, and this new address will persist beyond a reboot. Set the address as follows:

=> **setenv ethaddr** *AA:BB:CC:DD:EE:FF* **;saveenv**

WNDRVR

Parent topic: Network and Ethernet (MAC) Configuration on page 14

2. GPIO

The general purpose input/output is supported by this BSP, and is enabled by including the component **DRV_RCAR_H3_GPIO** in the VIP.

The application must use the VxBus GPIO API to access the driver functions.

GPIO is validated on the Renesas R-Car H3SiP System Evaluation boards Salvator-X RTP0RC7795SIPB0011S-S and Salvator-XS RTP0RC7795SIPB0012SA03. GPIO was validated on these boards using two additional hardware devices:

- LED4/5/6 connected to GPIO GP6_11/12/13 in output mode.
- tactile switch SW20/21/22 which is connected to GPIO GP6_11/12/13, configured in interrupt mode.

3. Timer

Timer Unit (TMU)

The public timers are contained in the TMU. Add this support to the VIP from the component **DRV_TIMER_FDT_TMU**. These timers can be used as system clock and auxiliary clock by this BSP.

ARM Generic Timer

This is the generic timer in the ARM Cortex-A7/A8 MPCore processor. Add support to the VIP using the component **DRV_ARM_GEN_TIMER**. This timer is used by the BSP as the timestamp clock.

4. I2C

I2C is supported by the BSP. To use it, add the component **DRV_RCAR_H3_I2C** to the VIP. The application must then use the VxBus I2C API to access the I2C function.

5. eMMC

eMMC is supported by the BSP. To use it, add the components **DRV_RCAR_H3_MMC** and **DRV_MMCSTORAGE_CARD** to the VIP.

Note: HS400 is supported by silicon version 2.0 and later. HS200 is supported by silicon version 1.0 and 1.1. Only HS200 is verified on silicon version 1.1.

6. USB

USB 2.0 and USB 3.0 host mode is supported in this BSP.

WNDRVR

To enable USB support, ensure that the USB layer is added to the VSB. For example:

```
C:\WindRiver\workspace\myVSB> vxprj vsb add USB
```

USB 2.0 host mode is enabled in the VIP by including the following components:

```
C:\WindRiver\workspace\myVIP> vxprj vip component add INCLUDE_OHCI_INIT
C:\WindRiver\workspace\myVIP> vxprj vip component add INCLUDE_EHCI_INIT
C:\WindRiver\workspace\myVIP> vxprj vip component add INCLUDE_USB_PHY_RCAR_GEN3
```

For USB 3.0 host mode support, configure the following additional VIP components:

```
C:\WindRiver\workspace\myVIP> vxprj vip component add INCLUDE_USB_XHCI_HCD_INIT
C:\WindRiver\workspace\myVIP> vxprj vip parameter set myVIP XHCI_RCAR_ENABLE TRUE
```

Note:

Enable appropriate USB class drivers to use different USB devices. See VxWorks 7 USB Host Stack Programmer's Guide for more information. For example, the following command adds the USB mass storage class driver to support USB mass storage devices.

```
C:\WindRiver\workspace\myVIP> vxprj vip component add INCLUDE_USB_GEN2_STORAGE_INIT
```

7. PCIe

PCIe is supported by this BSP and is included by adding the component **DRV_FDT_RCAR_H3_PCIE** to the VIP.

8. CPG and MSSR

The clock pulse generator (CPG) and module standby / software reset (MSSR) modules are supported by this BSP.

The CPG generates clocks for CPU cores and peripherals, while the MSSR clocks control peripheral modules.

To support this in the VIP add the component **DRV_CLK_FDT_CPG_MSSR**. This driver must be included to support all the VxBus drivers that require CPG or MSSR clock sources. When a driver requires this clock source, then it must be listed as a required clock source and parameter pair.

For example, i2c depends on these clock sources. In the file **renesas-rcar-h3.dtsi**:

```
        i2c0: i2c@e6500000
            {
            ...
            clocks = <&cpg 31>,                 /* S3D2 is I2C input clock */
                     <&cpg 189>;                /* I2C-IF0 module clock */
            clock-names = "inputClk", "moduleClk";
```

The i2c driver must enable the clock and get its frequency in the driver attach method:

```
    if (vxbClkEnableAll (pDev) != OK)
        {
        goto errHandle;
        }
    pClk = vxbClkGet (pDev, RCARH3_I2C_INPUT_CLOCK);
    if (pClk == NULL)
        {
        goto errHandle;
```

```
        }
    inputClkFreq = vxbClkRateGet (pClk);
```

9. PinMux

The PinMux module which supplies PinMux management functionality is supported in this release of BSP. It is supported by the component **DRV_PINMUX_FDT_PFC**, and is enabled by default.

The PinMux module provides several groups of PinMux setting values in the device tree nodes. Each group includes the whole PinMux setting of a specified device. The device driver needing the PinMux function must find a supported group handle and write its device tree source node.

For example, in **renesas-salvator-x.dts**, the UART references the PinMux group handle of uart1:

```
    scif1Pins: scif1
        {
        /* TX1_A, TX1_A, CTS1#, RTS1# */
        pins = "sipE38", "sipE39", "sipH38", "sipH39";
        function = "scif1";
        bias-disable;
        drive-control = <8>;
        };
```

The UART driver must then enable the PinMux in the probe or attach function:

```
    vxbPinMuxEnable (pDev);
```

6. KNOWN BSP ISSUES

1. PCIe Limitations

The PCIe controller relies on the PCIe adapter driver. When developing this driver, only the Intel Pro/1000 VxBus enhanced network driver was available and verified for use. The detailed hardware information is Intel Gigabit CT Desktop Adapter, WG82574L. To use this driver, add the component **INCLUDE_GEI825XX_VXB_END**.

2. SMP

VxWorks Symmetric Multiprocessing (SMP) is supported when this BSP is used on the supported boards. For details on VxWorks SMP, refer to the *VxWorks 7 Application Programmer's Guide: VxWorks SMP*.

The main issues relating to SMP on this BSP are:

1. MMU
   MMU-less configuration is unsupported.
2. Cache and Bus Snooping
   Cache cannot be disabled in an SMP system. Direct calls to cacheDisable( ) do nothing.
   Only copy-back mode is tested with this BSP. Changing the configuration to write-through mode may result in unpredictable behaviour.
3. CPU Number
   The VIP parameter VX_SMP_NUM_CPUS specifies the number of CPUs assigned to the SMP kernel. The CPU referred to includes threads, hyper-threads and so forth.
   VxWorks supports using less than the total number of CPUs in an SMP system. This permits some CPUs to be used for additional tasks such as dedicated tasks or additional asymmetric operating systems.
   CPUs not assigned to the SMP kernel will remain in the default, power-on reset state.

3. TSN Limitations

The **renesas_rcar_h3** BSP supports Precision Time Protocol (**PTP**) to synchronize clocks throughout your network. However, it does not support the **TSN** clocks and timers library (**tsnClkLib**) and the 802.1Qav and 802.1Qbv **TSN** library (**qav_qbv**) in VxWorks.

7. REFERENCES

1. See Also

- VxWorks 7 Platform Programmer's Guide
- VxWorks 7 Application Programmer's Guide
- VxWorks 7 Configuration and Build Guide
- Helix Platform Configuration and Build Guide
- VxWorks 7 ARM Architecture Guide
- VxWorks 7 BSP and Driver Guide
- Renesas R-Car Series, 3rd Generation User's Manual: Hardware
- Renesas R-CarH3-SiP System Evaluation Board Salvator-X Setup Manual
- Renesas R-CarH3-SiP System Evaluation Board Salvator-XS Setup Manual
- Renesas R-CarH3-SiP System Evaluation Board Salvator-X Hardware Manual
- Renesas R-CarH3-SiP System Evaluation Board Salvator-XS Hardware Manual
- Renesas Linux Interface Specification Yocto Recipe Start-Up Guide User's Manual: Software R-Car M3/M3 Series
- Renesas Initial Program Loader User's Manual: Software R-Car H3/M3 Series